

Zero-Shot Prompting

Day 2 — Notebook 1 of 5 | Estimated Time: 20 minutes

What You'll Learn

- What zero-shot prompting is and when to use it
- How to write clear, direct instructions
- Common zero-shot tasks: classification, extraction, summarization, translation

What is Zero-Shot Prompting?

Zero-shot means you give the model a task **without any examples**. You simply describe what you want, and the model uses its pre-trained knowledge to complete it.

This is the simplest form of prompting and works well for:

- Straightforward tasks the model already knows how to do
- When you don't have example data to provide
- Quick prototyping and exploration

Setup

```
%pip install -U -q "google-genai>=1.0.0"
```

```
!pip install google-auth==2.47.0
```

```
Requirement already satisfied: google-auth==2.47.0 in /usr/local/lib/python3.12/dist-packages (2.47.0)  
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (0.4.0)  
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (4.9.1)  
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->google-auth==2.47.0) (0.6.1)
```

```
import sys, os  
import time  
  
import json  
import re  
from google import genai  
from google.genai import types  
from IPython.display import Markdown  
  
try:  
    from google.colab import auth  
    auth.authenticate_user()  
    print("✅ Authenticated via Colab")  
except ImportError:  
    print("✅ Using local credentials (gcloud auth)")  
  
client = genai.Client(  
    vertexai=True,  
    project="chanakya-476010",  
    location="us-central1",  
)  
MODEL_ID = "gemini-2.5-flash"  
print("✅ Ready!")
```

```
✅ Authenticated via Colab  
✅ Ready!
```

1. Classification

Ask the model to classify text into categories — no examples needed.

```
# Sentiment classification
prompt = """
Classify the sentiment of this review as Positive, Negative, or Neutral:

"I go to this restaurant every week, I love it so much."
"""

response = client.models.generate_content(model=MODEL_ID, contents=prompt)
print(f"Sentiment: {response.text.strip()}")
```

Sentiment: Positive

```
# Topic classification
prompt = """
Classify the following news headline into one of these categories:
Technology, Sports, Politics, Entertainment, Science

Headline: "NASA's James Webb Space Telescope captures stunning images of distant galaxies"
"""

response = client.models.generate_content(model=MODEL_ID, contents=prompt)
print(f"Category: {response.text.strip()}")
```

Category: Science

▼ 2. Information Extraction

Extract specific information from unstructured text.

```
# Extract entities
prompt = """
Extract all person names, organizations, and locations from this text:

"Sundar Pichai, CEO of Google, announced a new AI research partnership
with Stanford University during a conference in San Francisco."
"""

response = client.models.generate_content(model=MODEL_ID, contents=prompt)
Markdown(response.text)
```

Here are the extracted entities:

- **Person Names:** Sundar Pichai
- **Organizations:** Google, Stanford University
- **Locations:** San Francisco

```
# Extract structured data
prompt = """
Extract the following details from this job posting:
- Job Title
- Company
- Location
- Required Experience

Job Posting: "We at TechCorp are looking for a Senior Data Scientist
in our Bangalore office. Candidates must have 5+ years of experience
in machine learning and Python."
"""

response = client.models.generate_content(model=MODEL_ID, contents=prompt)
Markdown(response.text)
```

Here are the extracted details:

- **Job Title:** Senior Data Scientist
- **Company:** TechCorp
- **Location:** Bangalore
- **Required Experience:** 5+ years in machine learning and Python

3. Summarization

Condense long text into shorter summaries.

```
prompt = """
Summarize the following text in 2 sentences:
```

```

"Artificial intelligence has been transforming numerous industries, from healthcare
to finance, manufacturing to entertainment. In healthcare, AI is being used for
drug discovery, medical imaging analysis, and patient diagnosis support. Financial
institutions use AI for fraud detection, algorithmic trading, and risk assessment.
The manufacturing sector has embraced AI for quality control, predictive maintenance,
and supply chain optimization. Meanwhile, the entertainment industry uses AI for
content recommendation, game development, and special effects creation."
"""
```

```
response = client.models.generate_content(model=MODEL_ID, contents=prompt)
Markdown(response.text)
```

Artificial intelligence is profoundly transforming numerous industries, including healthcare, finance, manufacturing, and entertainment. Across these sectors, AI is applied for diverse purposes such as drug discovery, fraud detection, predictive maintenance, and content recommendation.

4. Translation & Transformation

Convert text between languages, formats, or styles.

```
# Translation
prompt = """
Translate the following English text to French, Spanish, and Japanese:
```

```

"The weather is beautiful today. Let's go for a walk in the park."
"""
```

```
response = client.models.generate_content(model=MODEL_ID, contents=prompt)
Markdown(response.text)
```

Here's the translation of "The weather is beautiful today. Let's go for a walk in the park." into French, Spanish, and Japanese:

French: Il fait beau aujourd'hui. Allons nous promener au parc.

Spanish: El clima está hermoso hoy. Vamos a dar un paseo en el parque.

Japanese: 今日は素晴らしい天気です。公園に散歩に行きましょう。(Kyō wa subarashii tenki desu. Kōen ni sanpo ni ikimashō.)

```
# Code explanation
prompt = """
Explain what this Python code does in plain English:
```

```

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
"""
```

```
response = client.models.generate_content(model=MODEL_ID, contents=prompt)
Markdown(response.text)
```

This Python code calculates a specific number in the famous **Fibonacci sequence** using a technique called **recursion**. Here's how it works in plain English:

- What it is:** It defines a function called `fibonacci` that takes one input, which we'll call `n`. Think of `n` as saying "which" Fibonacci number you want to find (e.g., the 5th, the 10th).
- The Starting Points (Base Cases):**
 - The first part, if `n <= 1`: `return n`, handles the very beginning of the sequence.
 - If you ask for `fibonacci(0)`, it immediately gives you `0`.
 - If you ask for `fibonacci(1)`, it immediately gives you `1`.
 - These are the first two numbers in the standard Fibonacci sequence.
- The Recursive Step (How it builds up):**
 - If `n` is greater than 1 (meaning you're not asking for the very first two numbers), the function does something clever:
 - `return fibonacci(n-1) + fibonacci(n-2)`
 - It tells itself: "To find the `n`-th Fibonacci number, I need to know the Fibonacci number *just before it* (which is `n-1`) AND the Fibonacci number *two places before it* (which is `n-2`). Once I have those two, I'll add them together."
- How it "unwinds":**
 - This process keeps repeating. For example, if you ask for `fibonacci(5)`:
 - It asks for `fibonacci(4) + fibonacci(3)`.
 - To find `fibonacci(4)`, it asks for `fibonacci(3) + fibonacci(2)`.
 - To find `fibonacci(3)`, it asks for `fibonacci(2) + fibonacci(1)`.
 - This continues until it reaches `fibonacci(0)` or `fibonacci(1)`, for which it already knows the answer (0 and 1, respectively).
 - Then, it adds those basic answers back up the chain until it gets the final result for `fibonacci(5)`.

In short: This code calculates a Fibonacci number by defining the first two numbers directly, and then defining every subsequent number as the sum of the two numbers that came before it, repeatedly calling itself until it reaches those initial known values.

The Fibonacci sequence starts like this: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

5. Tips for Better Zero-Shot Prompts

Tip	Bad Example	Good Example
Be specific	"Tell me about dogs"	"List 5 popular dog breeds for families with children"
Specify format	"Summarize this"	"Summarize this in 3 bullet points"
Set constraints	"Write about AI"	"Write a 100-word explanation of AI for beginners"
Define the task	"Fix this"	"Find and fix the syntax error in this Python code"

🔗 Exercise 1: Classification Challenge

Write zero-shot prompts to classify the following customer support messages into categories: `Billing`, `Technical`, `General Inquiry`, or `Complaint`.

```
# Exercise 1: Classify customer support messages

messages = [
    "I was charged twice for my subscription last month.",
    "How do I reset my password?",
    "Your product is terrible and I want a full refund!",
    "What payment methods do you accept?",
    "The app keeps crashing when I try to upload a file.",
]

for msg in messages:
    # TODO: Write a prompt that classifies the message
    prompt = f"""Classify the prompts into meaningful buckets like Billing, Technical, General Inquiry, or Complaint.

    {msg}
    """

    response = client.models.generate_content(model=MODEL_ID, contents=prompt)
    print(f"Message: {msg}")
    print(f"Category: {response.text.strip()}")
    print()
```

Message: I was charged twice for my subscription last month.
Category: **Billing**

Message: How do I reset my password?
Category: **Technical**

Message: Your product is terrible and I want a full refund!

Category: ****Complaint****

Message: What payment methods do you accept?

Category: ****Billing****

Message: The app keeps crashing when I try to upload a file.

Category: ****Technical****

🔧 Exercise 2: Email Extractor

Write a zero-shot prompt that extracts the sender, recipient, subject, urgency level, and key action items from an email.

```
# Exercise 2: Email information extraction
```

```
email = """
From: sarah.chen@company.com
To: dev-team@company.com
Subject: URGENT - Production database migration this Friday
```

```
Hi team,
```

```
This is a reminder that we have a critical database migration scheduled
for this Friday at 10 PM EST. Please ensure all pending PRs are merged
by Thursday EOD. The migration window is 4 hours, and we need at least
two engineers on-call during this time.
```

```
Action items:
```

1. Merge all pending PRs by Thursday
2. Confirm your on-call availability
3. Run the pre-migration checklist

```
Thanks,
Sarah
"""
```

```
# TODO: Write a prompt that extracts structured info from this email
```

```
prompt = f"""Give me a structured format of the main details like sender,recipient , issue , time , Action items in a json for
then will be needed for an MoM. Have no more than 3 important details per field
{email}
"""
```

```
response = client.models.generate_content(model=MODEL_ID, contents=prompt)
Markdown(response.text)
```

```
{
  "sender": {
    "name": "Sarah Chen",
    "email": "sarah.chen@company.com"
  },
  "recipient": {
    "name": "Dev Team",
    "email": "dev-team@company.com"
  },
  "issue": {
    "title": "URGENT: Production database migration",
    "status": "Critical reminder",
    "context": "Scheduled event"
  },
  "time": {
    "migration_start": "Friday, 10 PM EST",
    "pr_deadline": "Thursday EOD",
    "migration_window": "4 hours"
  },
  "action_items": [
    "Merge all pending PRs by Thursday",
    "Confirm on-call availability",
    "Run pre-migration checklist"
  ]
}
```

📖 Further Reading

Resource	Type	Link
Zero-Shot Prompting Guide	Blog	promptingguide.ai/techniques/zeroshot
Google Gemini Prompting Strategies	Docs	ai.google.dev/gemini-api/docs/prompting-strategies

Next up: [02_Few_Shot_Prompting.ipynb](#) — Teach the model by providing examples